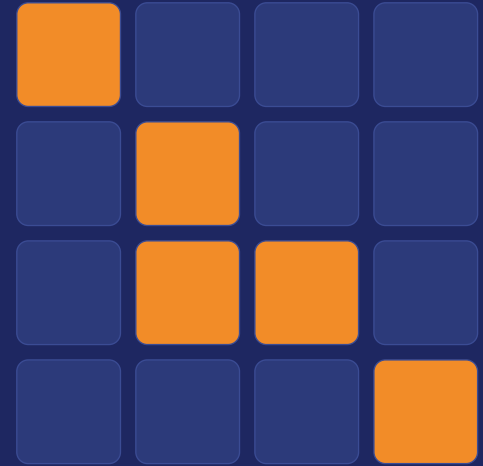


LECTURE 6

Pairwise Alignment Algorithms

Needleman–Wunsch · Smith–Waterman
Dynamic programming, traceback, complexity

Bioinformatics · MS Data Science



SHAHID MANOOR

Learning objectives

1

Explain

the Needleman–Wunsch and
Smith–Waterman algorithms

2

Construct

a dynamic-programming
scoring matrix

3

Perform

traceback to recover an
optimal alignment

Warm-up poll: which alignment is best?

Align GATTA (top) with GCTA (bottom). Score: match +1, mismatch -1, gap -2

A

GATTA
GCT-A

B

GATTA
G-CTA

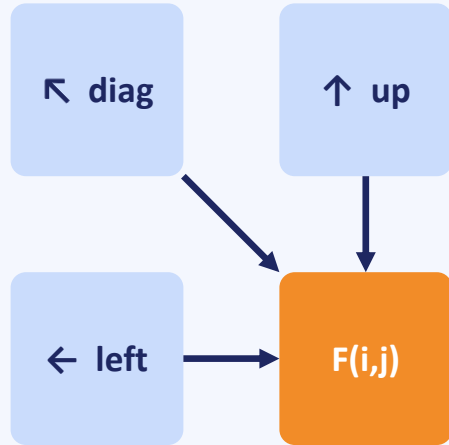
C

GATTA
GCTA-

Show of hands

1. Vote A, B or C (30 s)
2. Turn to a neighbour and defend your pick
3. Score all three on paper — did anyone change their vote?

The idea: build the answer cell by cell



↖ align two letters

↑ gap in top sequence ← gap in bottom sequence

Needleman-Wunsch recurrence

$$F(i,j) = \max \text{ of}$$
$$F(i-1,j-1) + s(a_j, b_i)$$
$$F(i-1,j) + \text{gap}$$
$$F(i,j-1) + \text{gap}$$

Start: $F(0,j) = j \times \text{gap}$, $F(i,0) = i \times \text{gap}$

Scores used today: match +1, mismatch -1, gap -2

Answer = bottom-right cell

Quick check: compute one cell

1	-1
-1	?

The letters being compared are A and C.

With match +1, mismatch -1, gap -2:
what value goes in the “?” cell?

Think alone 1 min, then pair up and compare.

Hint: compute all three candidates, then keep the largest.

Worked example and traceback

	G	A	T	T	A	
G C T A	0	-2	-4	-6	-8	-10
	-2	1	-1	-3	-5	-7
	-4	-1	0	-2	-4	-6
	-6	-3	-2	1	-1	-3
	-8	-5	-2	-1	0	0

Orange = traceback path

1. Start at the bottom-right cell
2. Step to the neighbour that produced the value
3. Diagonal = align letters; up/left = gap
4. Stop at the top-left cell

G A T T A
G - C T A

Score 0. Warm-up answer: A and B both score 0, C scores -2. Ties = several optimal paths.

Activity 1: fill the matrix by hand

	A	C	G	T	
A	0	-2	-4	-6	-8
	-2				
	-4				
	-6				

Pairs · 8 minutes

Top: ACGT Left: AGT

Match +1, mismatch -1, gap -2

1. Copy the recurrence on paper
2. Fill row by row, left to right
3. Put a small arrow in each cell showing which neighbour won
4. Raise your hand when you reach the corner

Activity 2: trace back as a team

1

Reader

calls out the cell value

2

Pointer

follows the arrow with a pencil

3

Scribe

writes the alignment, right to left

Teams of 3 · 6 minutes

Start at the bottom-right of your Activity 1 matrix and rebuild the alignment.

Stretch question: is there a second optimal path? Where does it branch?

Check your answer

	A	C	G	T	
A	0	-2	-4	-6	-8
G	-2	1	-1	-3	-5
T	-4	-1	0	0	-2
	-6	-3	-2	-1	1

Optimal alignment

A C G T

A - G T

Score: 1

Compare with your team. Did you get the same letters?

Did anyone find a different path with the same score?

Global vs local: Needleman–Wunsch vs Smith–Waterman

Needleman–Wunsch

Global

- Aligns whole sequences end to end
- Cells can be negative
- Traceback: bottom-right to top-left
- Best for similar-length, related sequences

Smith–Waterman

Local

- Finds the best matching region
- Scores floor at 0: $\max(0, \dots)$
- Traceback: highest cell to first 0
- Best for domains, motifs, partial matches

Three changes turn NW into SW: add a 0 option, initialise edges to 0, start traceback at the maximum cell.

Time and space complexity

$O(m \times n)$

Time: one constant-time step
per cell

$O(m \times n)$

Space: the full matrix is stored

$O(m+n)$

Traceback: one step per letter

1,000 × 1,000 letters = 1 million cells (fine)

10,000 × 10,000 letters = 100 million cells (memory becomes the limit)

Trick for long sequences: Hirschberg's algorithm keeps time $O(m \times n)$ but space $O(\min(m, n))$.

Lab and assignment

Lab: Needleman–Wunsch in Python

```
def needleman_wunsch(a, b, match=1,
                     mismatch=-1, gap=-2):
    n, m = len(b), len(a)
    F = [[0]*(m+1) for _ in range(n+1)]
    # 1. initialise first row/column
    # 2. fill with max(diag, up, left)
    # 3. traceback from F[n][m]
    return aligned_a, aligned_b, F[n][m]
```

Assignment

Implement Smith–Waterman and compare its output and runtime with Needleman–Wunsch on the same inputs.

Test on pairs of increasing length and report what you observe.

Takeaways and exit ticket

- 1 A DP matrix breaks alignment into tiny local decisions
- 2 Traceback follows the winning neighbours back to the start
- 3 NW is global, SW is local; both cost $O(m \times n)$

Exit ticket (1 min): In one sentence, why does Smith–Waterman never produce a negative score?